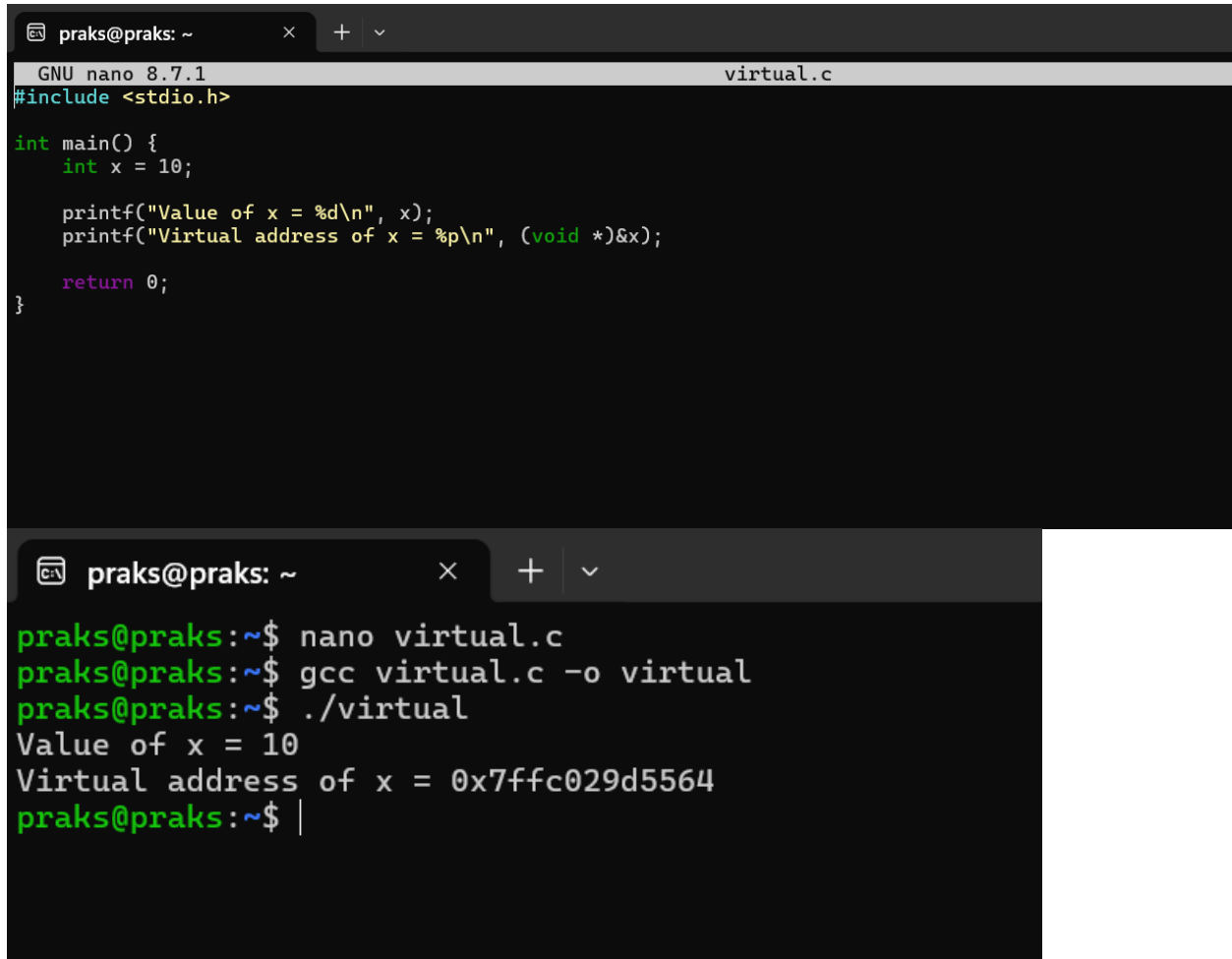


CODE 1:



The image shows a terminal window with two panes. The top pane displays the contents of a C file named `virtual.c` using the `nano` editor. The code defines a `main` function that declares an integer `x` with the value 10, prints its value and its memory address, and then returns 0. The bottom pane shows the execution of the program: the user runs `nano virtual.c` to edit the file, `gcc virtual.c -o virtual` to compile it, and `./virtual` to run it. The output of the program is printed to the terminal: "Value of x = 10" followed by "Virtual address of x = 0x7ffc029d5564".

```
praks@praks: ~  
GNU nano 8.7.1 virtual.c  
#include <stdio.h>  
  
int main() {  
    int x = 10;  
  
    printf("Value of x = %d\n", x);  
    printf("Virtual address of x = %p\n", (void *)&x);  
  
    return 0;  
}  
  
praks@praks: ~  
praks@praks:~$ nano virtual.c  
praks@praks:~$ gcc virtual.c -o virtual  
praks@praks:~$ ./virtual  
Value of x = 10  
Virtual address of x = 0x7ffc029d5564  
praks@praks:~$ |
```

CODE 2:

```
praks@praks:~$ nano adressspace.c
praks@praks:~$ gcc adressspace.c -o adressspace
praks@praks:~$ ./adressspace
Code address   : 0x5882245f81a9
Data address   : 0x5882245fb010
BSS address    : 0x5882245fb018
Heap address   : 0x5882633b2010
Stack address  : 0x7fff6c23839c
praks@praks:~$
```

```
praks@praks: ~
GNU nano 8.7.1                                adressspace.c *
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {

    int local = 20;

    int *heap = malloc(sizeof(int));
    if (heap == NULL) {
        return 1;
    }
    *heap = 30;

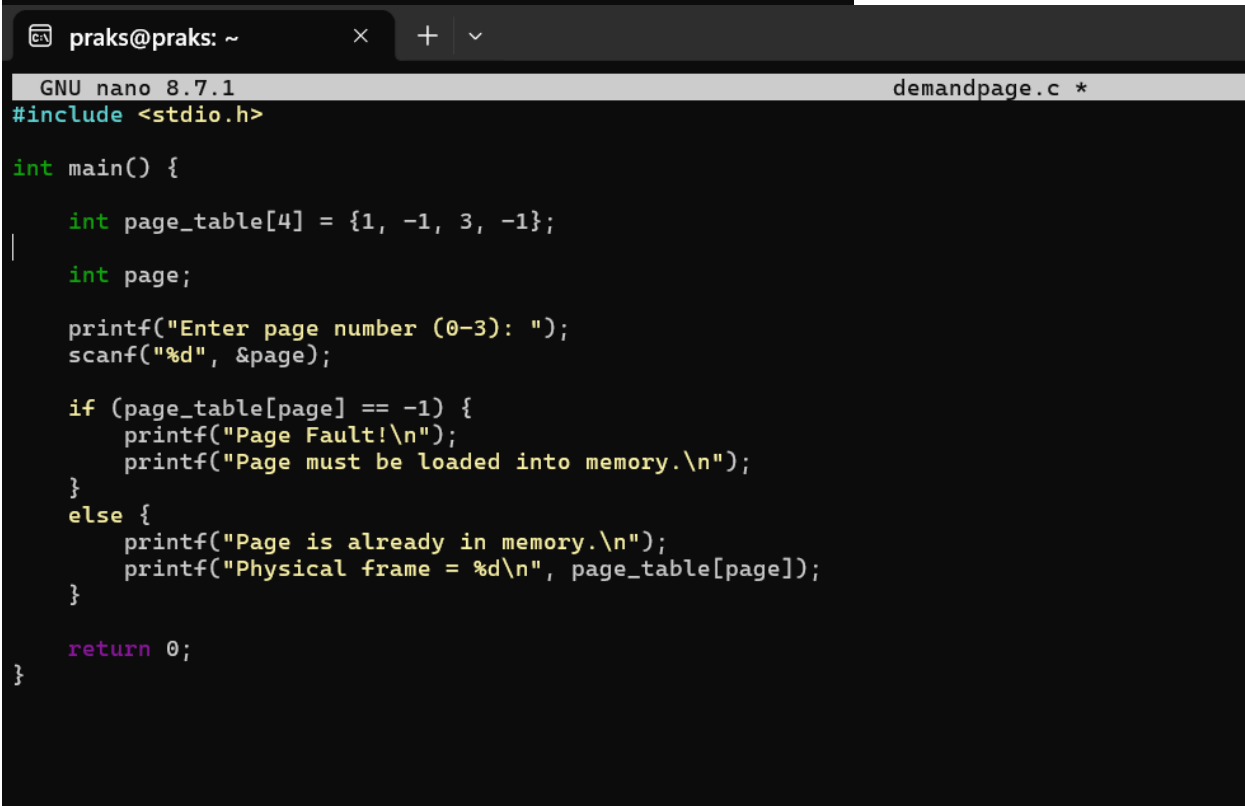
    printf("Code address   : %p\n", (void *)main);
    printf("Data address   : %p\n", (void *)&global);
    printf("BSS address    : %p\n", (void *)&bss_variable);
    printf("Heap address   : %p\n", (void *)heap);
    printf("Stack address  : %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

CODE 3:

```
praks@praks:~$ nano demandpage.c
praks@praks:~$ gcc demandpage.c -o demandpage
praks@praks:~$ ./demandpage
Enter page number (0-3): 1
Page Fault!
Page must be loaded into memory.
praks@praks:~$ |
```



The screenshot shows a terminal window with a nano editor editing a file named demandpage.c. The code defines a page_table array with values {1, -1, 3, -1} and a variable page. It prompts the user to enter a page number (0-3). If the value is -1, it prints "Page Fault!" and "Page must be loaded into memory.". Otherwise, it prints "Page is already in memory." and the physical frame number.

```
GNU nano 8.7.1 demandpage.c *
#include <stdio.h>

int main() {
    int page_table[4] = {1, -1, 3, -1};
    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

CODE 4:

```
praks@praks:~$ nano addtrans.c
praks@praks:~$ gcc addtrans.c -o addtrans
praks@praks:~$ ./addtrans
Virtual Address = 250
Page Number     = 2
Offset          = 50
Frame Number    = 1
Physical Address = 150
praks@praks:~$ |
```

```
praks@praks: ~  
GNU nano 8.7.1 addtrans.c *  
#include <stdio.h>  
  
int main() {  
    int page_table[] = {3, 5, 1, 7};  
    int page_size = 100;  
    int virtual_address = 250;  
    int page = virtual_address / page_size;  
    int offset = virtual_address % page_size;  
    int frame = page_table[page];  
    int physical_address = frame * page_size + offset;  
    printf("Virtual Address = %d\n", virtual_address);  
    printf("Page Number      = %d\n", page);  
    printf("Offset          = %d\n", offset);  
    printf("Frame Number     = %d\n", frame);  
    printf("Physical Address = %d\n", physical_address);  
    return 0;  
}
```

CODE 5:

```
praks@praks:~$ nano memory.c  
praks@praks:~$ gcc memory.c -o memory  
praks@praks:~$ ./memory  
10 20 30 40 50  
praks@praks:~$ |
```

5:

```
praks@praks: ~  
GNU nano 8.7.1 memory.c *  
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    int *p;  
  
    p = malloc(5 * sizeof(int));  
  
    if (p == NULL) {  
        printf("Memory allocation failed\n");  
        return 1;  
    }  
  
    for (int i = 0; i < 5; i++) {  
        p[i] = (i + 1) * 10;  
    }  
  
    for (int i = 0; i < 5; i++) {  
        printf("%d ", p[i]);  
    }  
  
    printf("\n");  
  
    free(p);  
  
    return 0;  
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U U
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E F

CODE 6:

```
praks@praks:~$ nano segmentationfault.c  
praks@praks:~$ gcc segmentationfault.c -o segmentationfault  
praks@praks:~$ ./segmentationfault  
Before accessing memory  
Segmentation fault                ./segmentationfault  
praks@praks:~$ |
```

```
praks@praks: ~  
GNU nano 8.7.1 segmentationfault.c *  
#include <stdio.h>  
  
int main() {  
    int *p = NULL;  
    printf("Before accessing memory\n");  
    *p = 100;  
    printf("After accessing memory\n");  
    return 0;  
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo